

CIVENTRAL Urban Planning

System Guide

1. What it is

A web app an LGU (local government unit) uses to manage residents, socialized housing, field inspections, and urban planning/permitting. Two user types:

- **Staff** - log in via the city's shared central staff account system (this project doesn't own that login).
- **Citizens** - a separate, local self-service account (register, upload documents, check application status), also used by a companion mobile app.

No install required - server-rendered PHP, opened in a browser.

2. Modules

Module	What it does
Resident Management	Master registry of residents and households. Every other module references it instead of duplicating identity data.
Housing Management	Unit lifecycle: vacant, applicant scored for eligibility, awarded, occupied, relocated if needed.
Field Survey	Form templates, assignments (resident/household/site), results. Recording a result auto-closes the assignment.
Urban Planning	Development plans, zoning clearances (auto-checked against a regulation matrix, generates certificates), urban projects, infrastructure records, permit review.
Barangay/GIS Mapping	Interactive map: barangay boundaries, subdivisions, building footprints (Leaflet).
Reports	Filterable/exportable/printable views over the other modules, no separate data entry.
AI Planning Assistant	Gemini-backed chat, grounded in a live analytics snapshot so answers use real numbers.
Citizen self-service	Local citizen accounts: registration, document upload, status checks, read-only housing programs/projects.

3. Architecture, briefly

- **Stack:** PHP (no framework), MySQL, Tailwind v4 via CDN, vanilla JS. No build step, no bundler.
- **Auth split:** staff auth/RBAC is proxied to a production system (civentral.tech) - this repo has no staff password table. Citizen accounts are local, bcrypt-hashed.

- **Two API patterns:** remote-proxy endpoints (forward to production) vs. local-DB endpoints (repository -> service -> API, own MySQL tables). See CLAUDE.md for the full pattern.
- **External services:** Gemini API (AI assistant), SMTP/PHPMailer (email), Google reCAPTCHA (login), OpenStreetMap Overpass (one-off map data import, not runtime).
- **Mobile:** separate sibling repo (Civentral-CitizenMobile, Expo/React Native), consumes the citizen-facing API over session cookies.

4. Security

Auth & sessions

- Staff credentials verified by the production system, never stored here. Citizen passwords hashed with bcrypt (password_hash/password_verify).
- 30-minute idle session timeout, enforced on every request.
- Citizen login: logout after 5 failed attempts (15 min). reCAPTCHA v2 on staff login, only enforced when site/secret keys are actually configured.

Authorization

- RBAC comes from the production system (granted_resources/granted_actions on the session). This project only gates UI/API access on it, no local permission table.
- Permission lookups cached 30s to cut repeated round-trips to the production API.

Data protection

- Secrets (DB_*, SMTP, reCAPTCHA, Gemini key) live in a gitignored .env, never committed.
- File uploads: MIME/extension whitelist, 5MB cap, randomized stored filenames, uploads/.htaccess disables script execution in upload dirs.
- Output escaping fixed for a stored-XSS class of bug (Aug 2026) - server-sourced text is escaped before innerHTML rendering.
- Session cookie: HttpOnly + SameSite. Non-web directories blocked from direct HTTP access.
- 100% PDO prepared statements - no raw SQL string interpolation found in the local-DB layer.

Known gaps (tracked, not hidden)

- No CSRF token - state-changing requests rely on same-origin cookies only.
- config/proxy.php calls the production API with CURLOPT_SSL_VERIFYPEER => false (TLS encryption still applies; certificate identity isn't checked).
- PermissionMiddleware::requireResource() is fallback-permissive: an authenticated user with no matching resource grant is still let through. Practically, a dedicated resource mostly governs sidebar/button visibility, not a hard API-level deny, for modules riding a shared grant.

5. Setup

Local (dev)

1. XAMPP + Composer.

2. Copy project into htdocs, composer install.
3. Create a DB, copy .env.example to .env, fill in values.
4. Run database/*.sql in the order listed in setup.md.
5. Browse to the local URL, log in with a production-provisioned account.

Deployment (Docker/Dokploy)

1. Dockerfile (PHP 8.2 + Apache) builds the image.
2. docker/entrypoint.sh waits for the DB, then docker/run-migrations.php applies all database/*.sql (tracked via a schema_migrations table so reddeploys don't replay them). dev_data_snapshot.sql is excluded from this auto-run.
3. Env vars set in the hosting platform (mirrors .env.example).
4. Mount a persistent volume at /var/www/html/uploads - otherwise uploaded files are lost on redeploy.

Full commands: setup.md (section 9 for deployment).

6. Further reading

- **setup.md** - exact setup/deploy commands.
- **docs/module-guide.md** - module walkthrough and cross-module flow.
- **docs/technical-documentation.md** - full engineering reference.
- **docs/activity-log.md** - chronological build log.
- **CLAUDE.md** - working conventions and architecture patterns.

7. Glossary

API - HTTP endpoint the frontend calls to read/write data.

RBAC - role-based access control; permissions tied to a role, not a person.

Session cookie - identifies a logged-in browser across requests.

Proxy - this project forwarding a request to the production system instead of handling it locally.

XSS - injected text executed as code on another user's page; prevented by escaping output.

CSRF - a third-party site tricking a logged-in browser into submitting an unwanted request.

Hashing - one-way transform of a password; not reversible, even by the system.

Docker - packages the app + runtime into one portable image.